

We are starting at **13:00!**

Grab a seat and get ready



Saturdays.AI
Kigali

#10 Modern CNNs

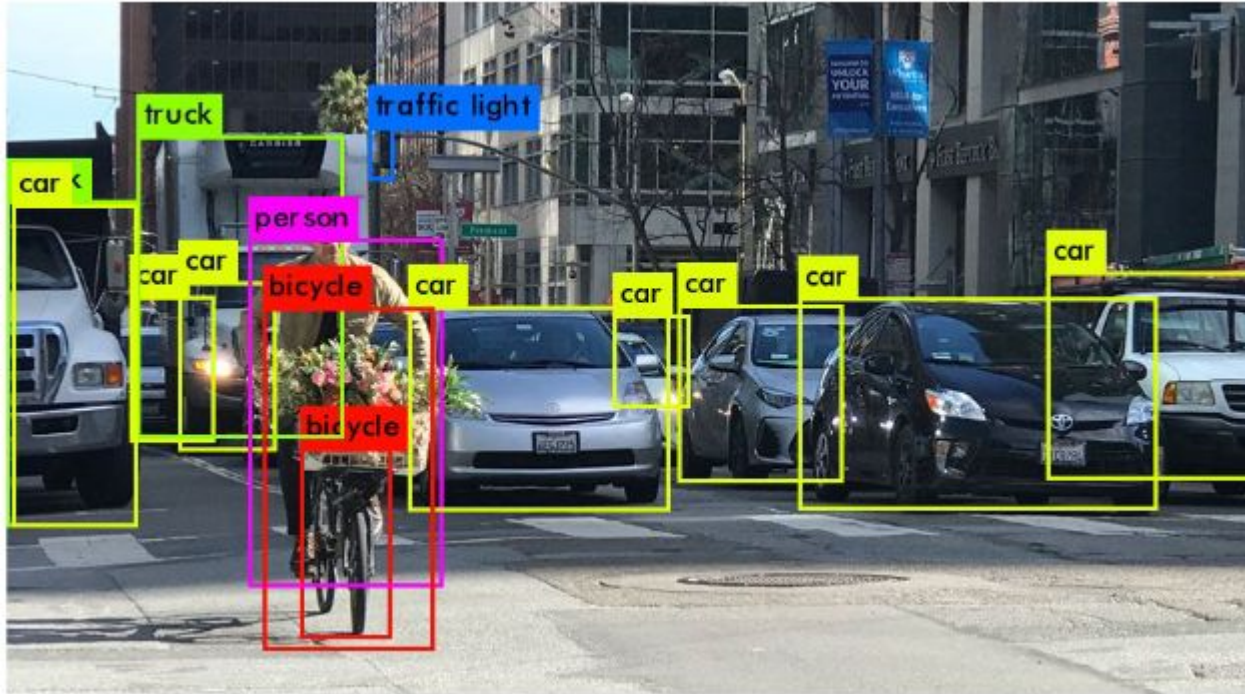
AI Saturdays Kigali

Last week: You learned to build a CNN

- Convolution
- Pooling/downsampling
- Training



Today: CNNs “in the real world”



CNNs are everywhere: image segmentation



CNNs are everywhere: pose estimation



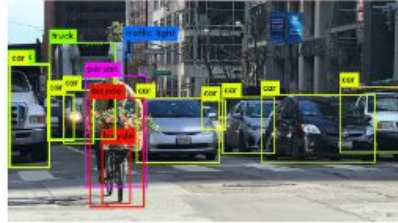
CNNs are everywhere: image style transfer



Today: Large-scale CNNs & transfer learning

The success of deep learning for image recognition has been driven by two key factors:

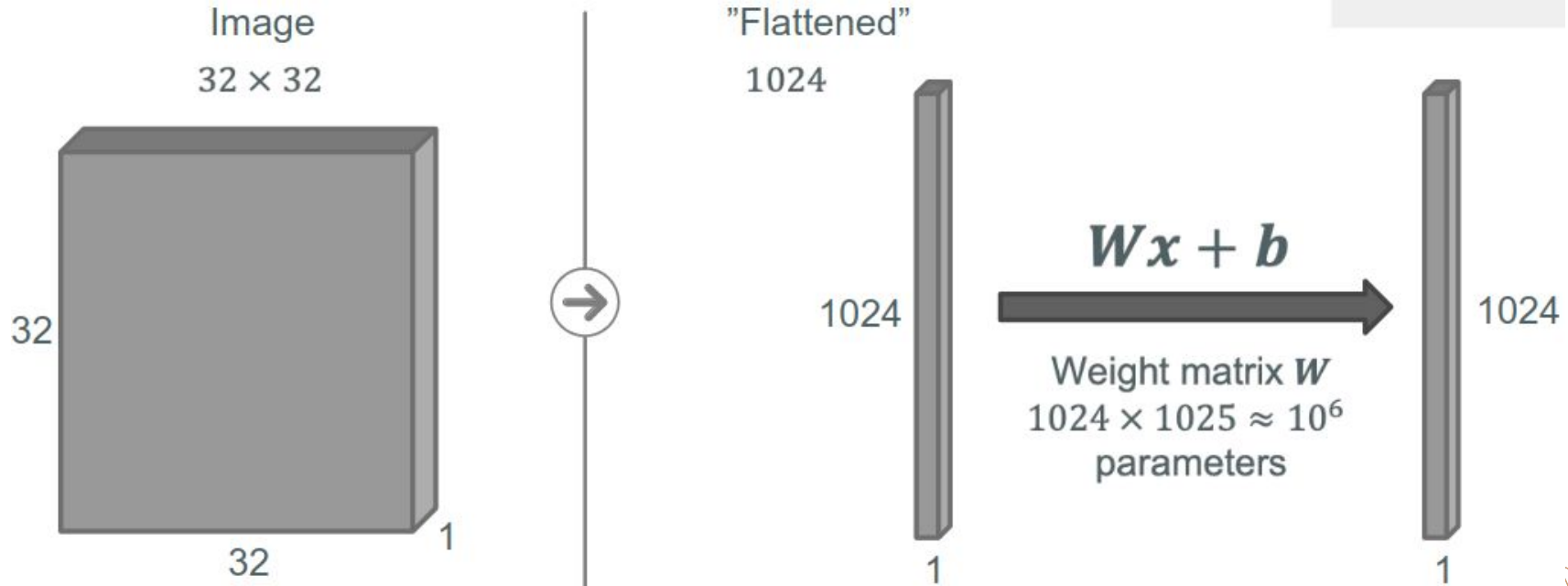
1. Large-scale CNNs
2. Transfer learning



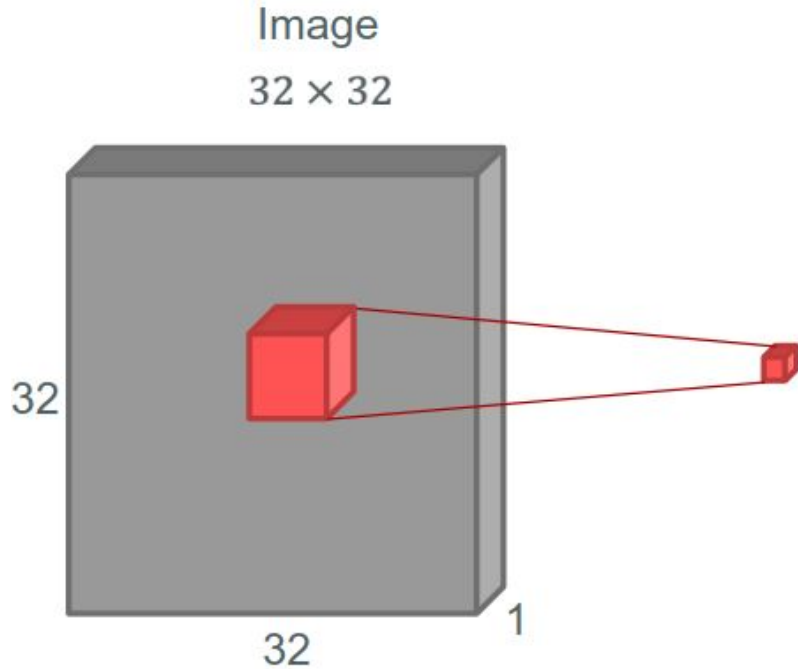
Recap: convolution



32 x 32 grayscale image: fully-connected layer



Convolution



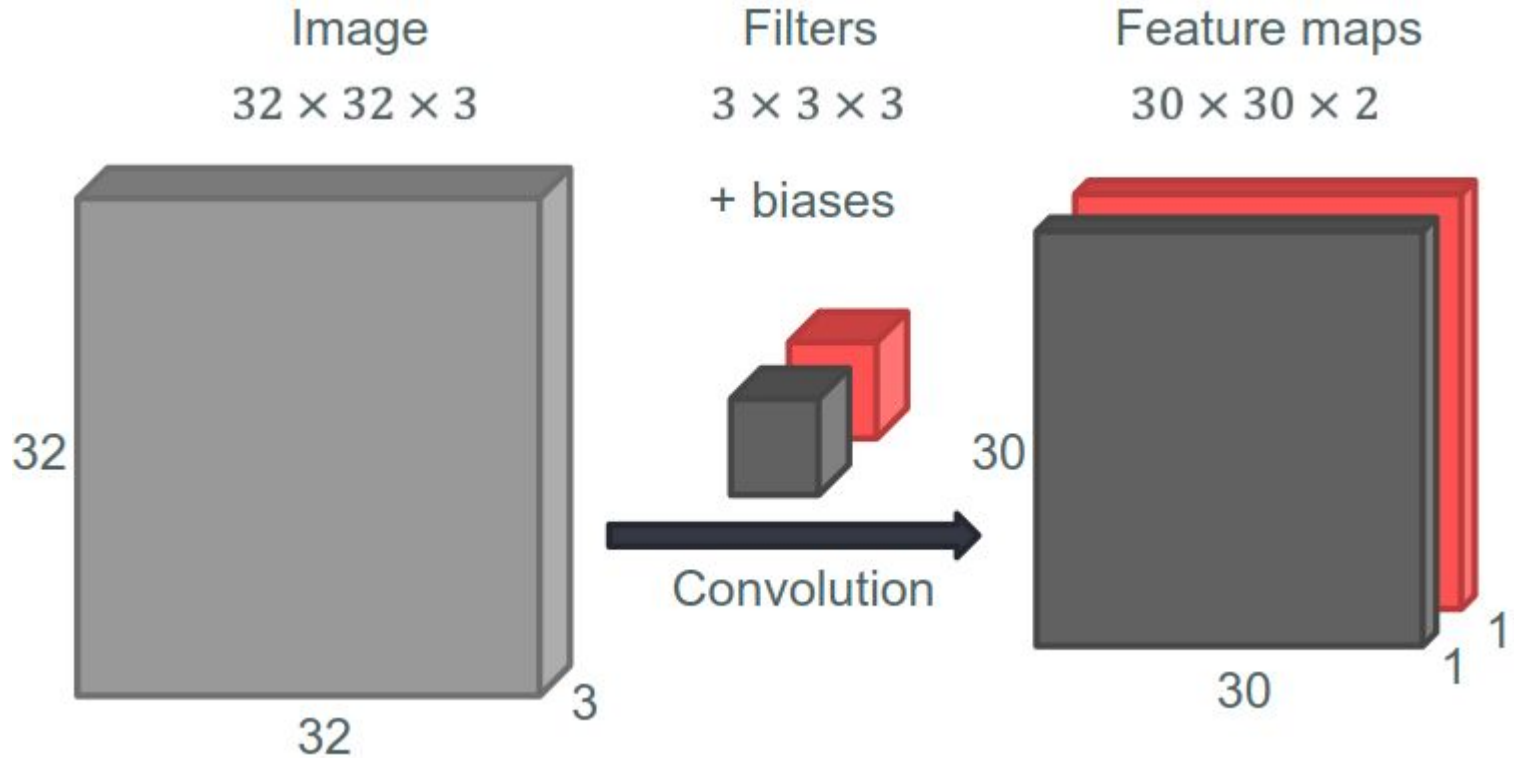
Convolve the filter with the image:
Slide over the image, computing dot products

1 number: $w^T x + b$

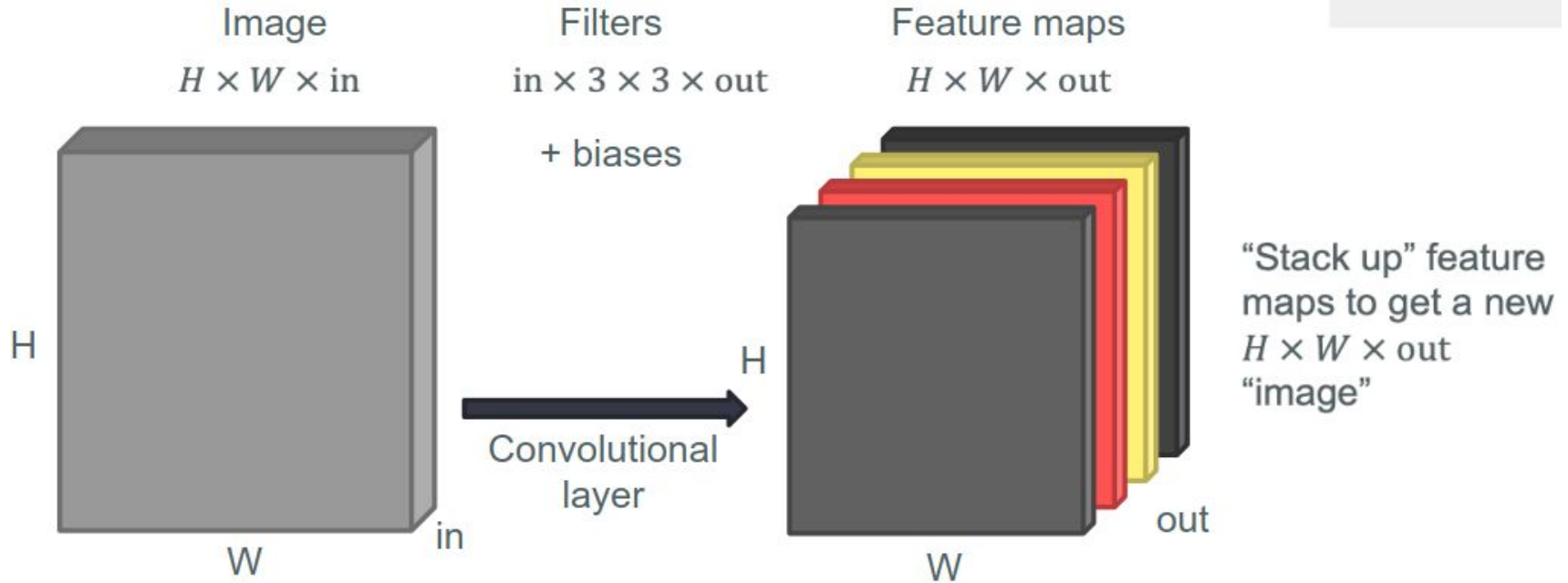
The result is the inner product of

- a local 3×3 chunk x of the image
- a 3×3 filter w
- plus a bias b

Convolution: color image



Convolution: in general

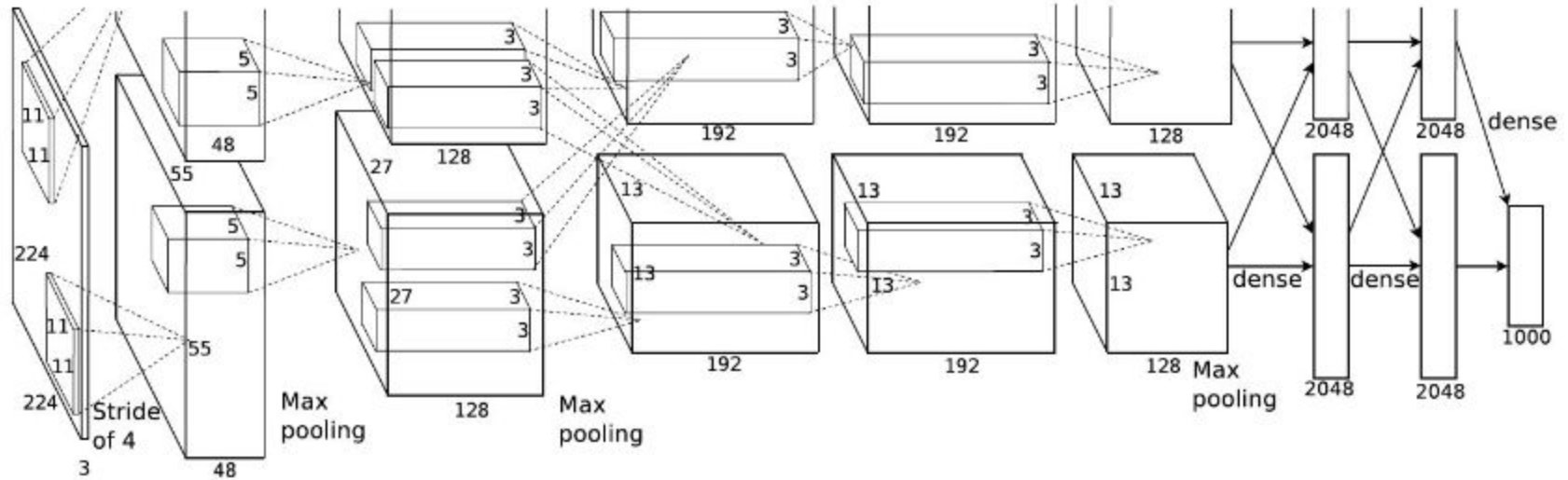


AlexNet & VGG



Saturdays.AI
Kigali

AlexNet (2012)

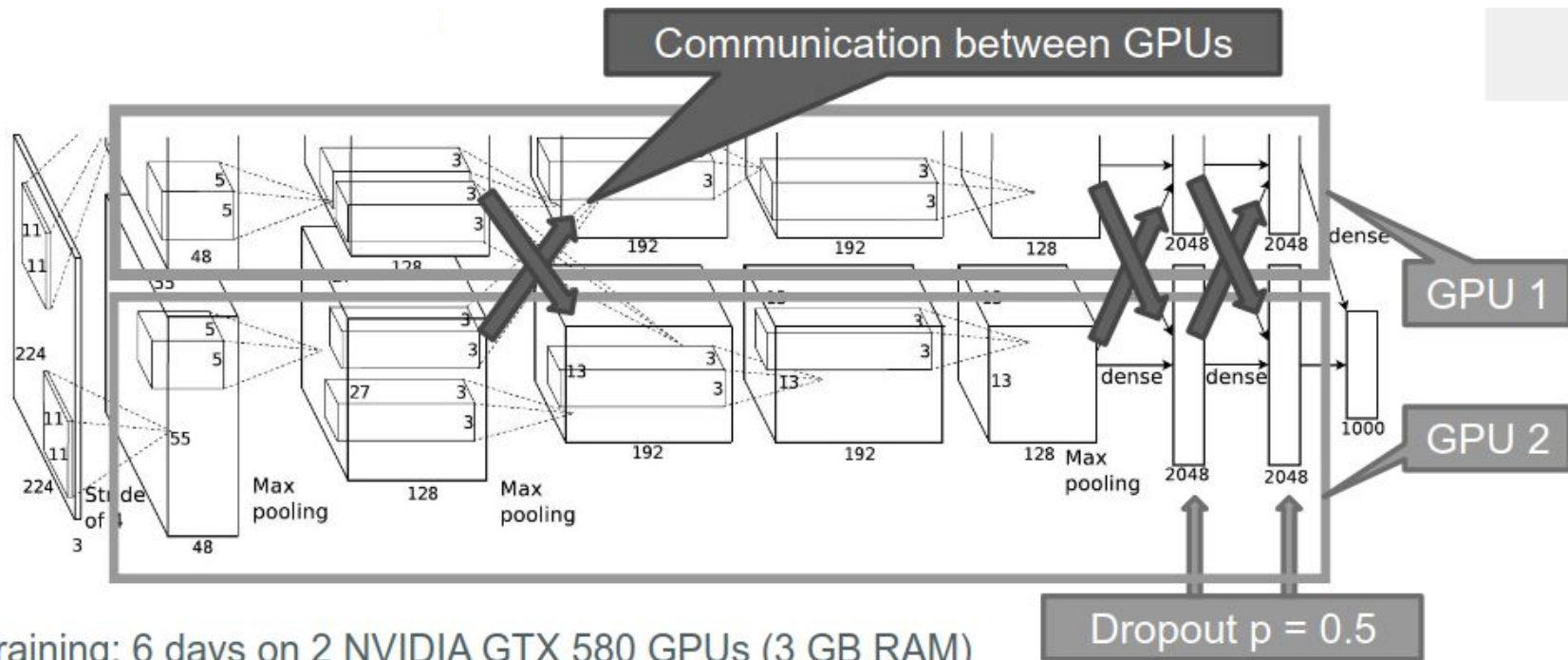


Alex Krizhevsky, Ilya Sutskever, Geoffrey Hinton

“ImageNet classification with deep convolutional neural networks.” NeurIPS 2012

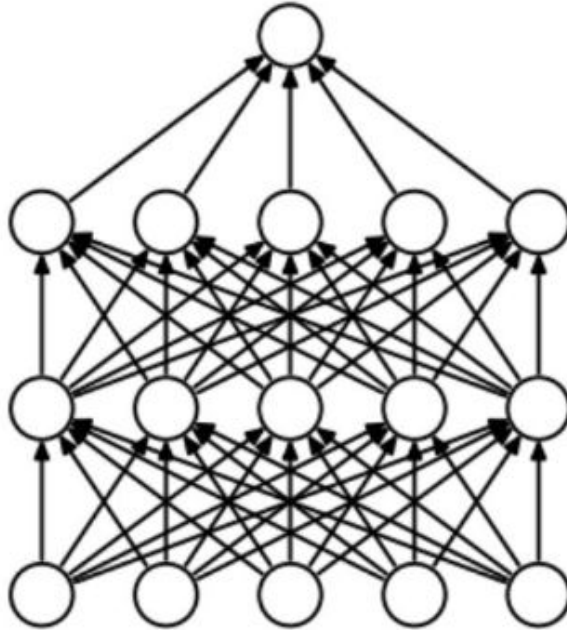
AlexNet (2012)

```
torchvision.models.alexnet()
```

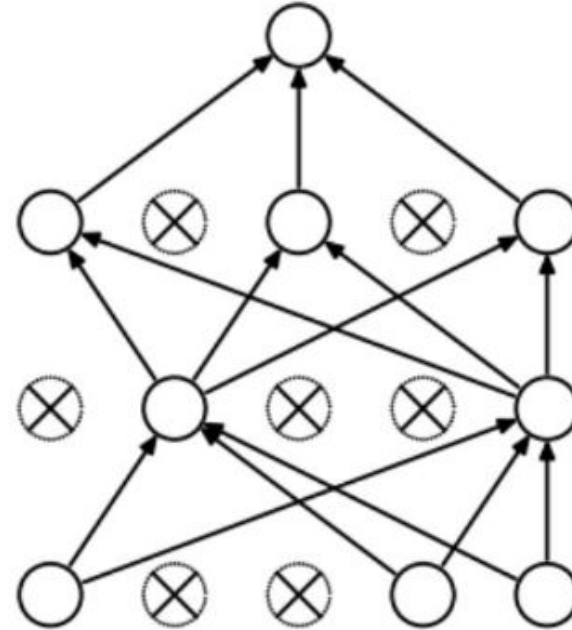


Training: 6 days on 2 NVIDIA GTX 580 GPUs (3 GB RAM)
8 layers / 60 million parameters

Dropout stochastically deactivates neurons

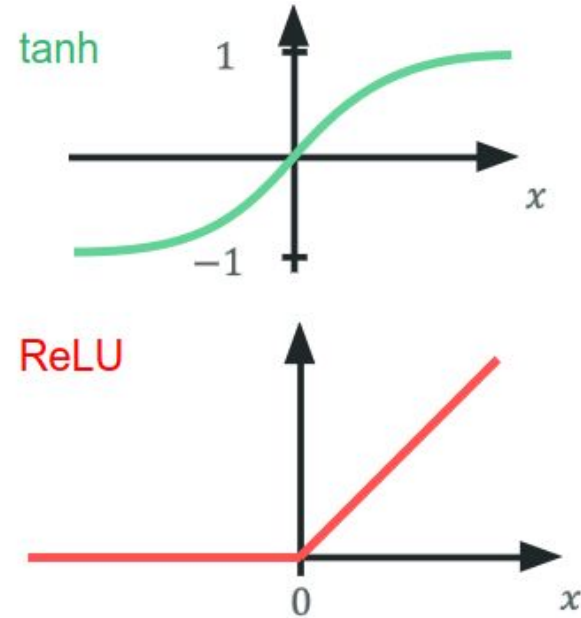
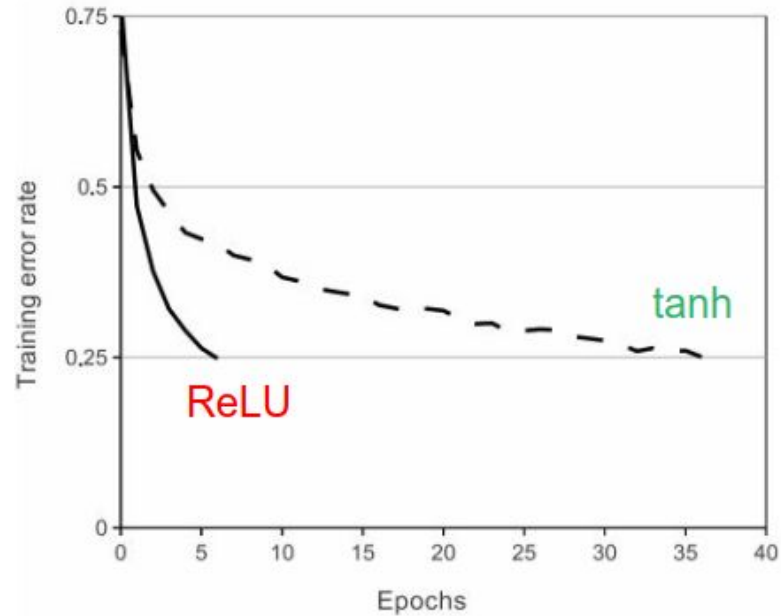


(a) Standard Neural Net



(b) After applying dropout.

ReLU trains faster than tanh

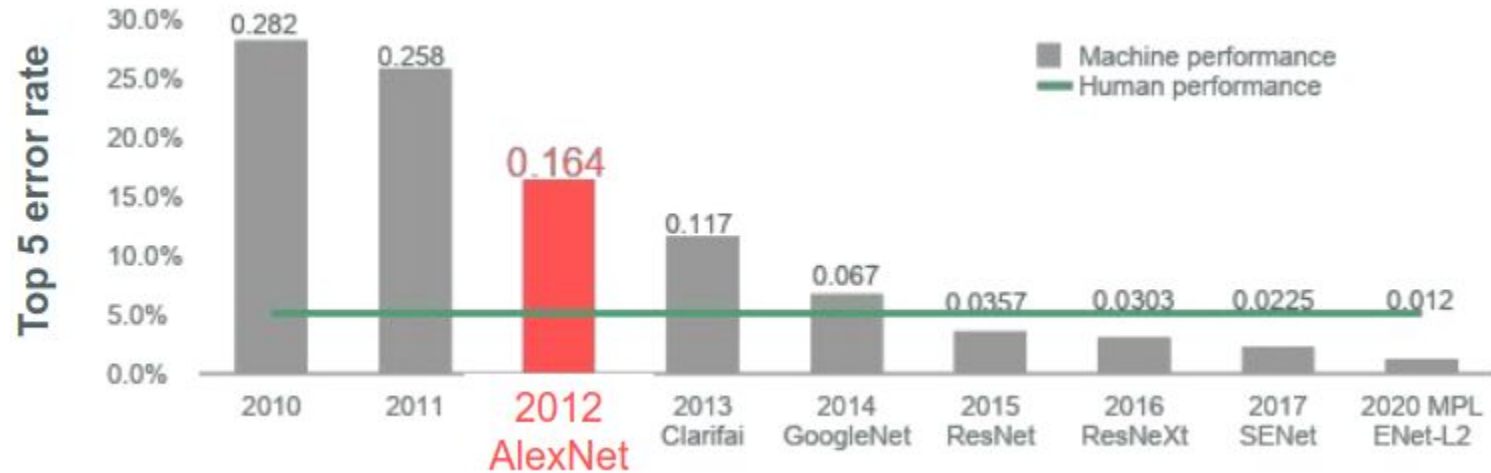


AlexNet (2012): Results

Smoked the competition with 16% top-5 error (runner-up had 26%)

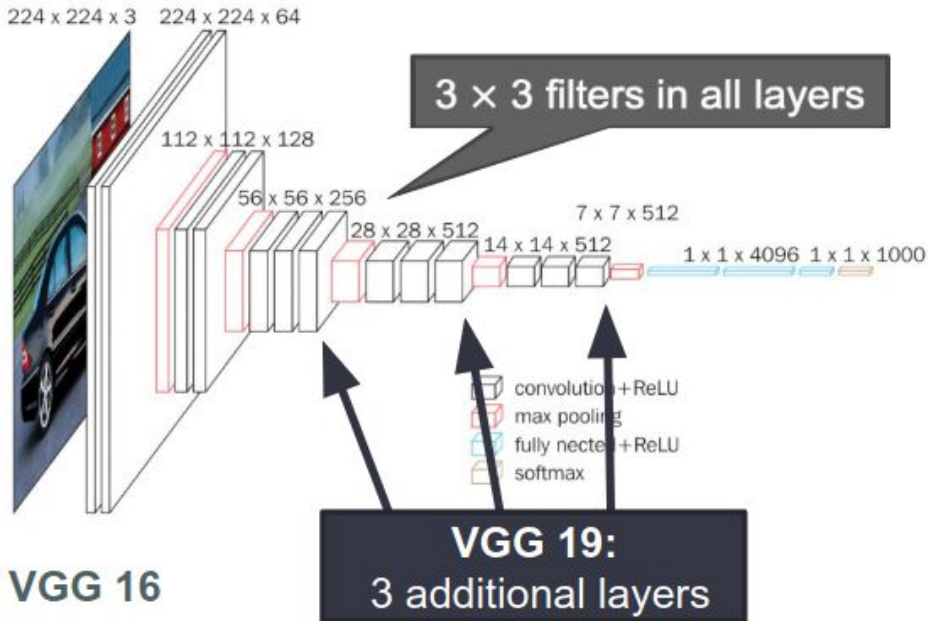
One of the first neural nets trained on a GPU with CUDA.

Paper cited over 80,000 times



VGG (2014)

```
torchvision.models.vgg16()  
torchvision.models.vgg19()  
torchvision.models.vgg19_bn()  
...
```



By the Vision Geometry Group at Oxford

Only 3×3 filters and max pooling

Training: 3 weeks on 4 NVIDIA Titan Black GPUs, 6 GB RAM

First train smaller configurations, then inject layers in between:

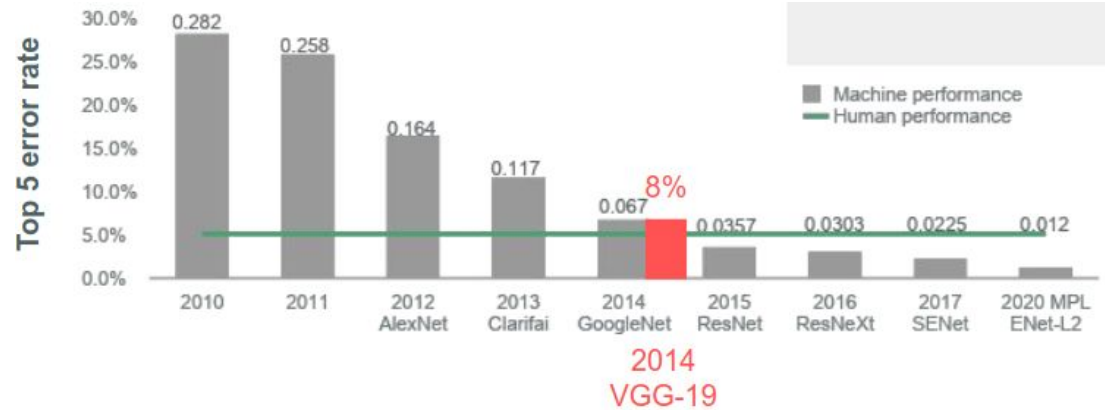
11 □ 13 □ 16 □ 19 layers

VGG-19: 138 million parameters / 500 MB

VGG (2014)

Has been very popular for a long time

- Particularly simple architecture
- Half the error rate of AlexNet
- Pre-trained net available



A couple of drawbacks:

- Slow and complex to train. Started with 11-layer version, then add additional layers, iterate until at 19
- Weights themselves are large: over 530 MB, which makes deploying it a tiresome task
- First couple of layers are very expensive to compute as they operate at full resolution

Interactive demo

*Inspect what AlexNet has learned:
filters and feature maps*

i 31

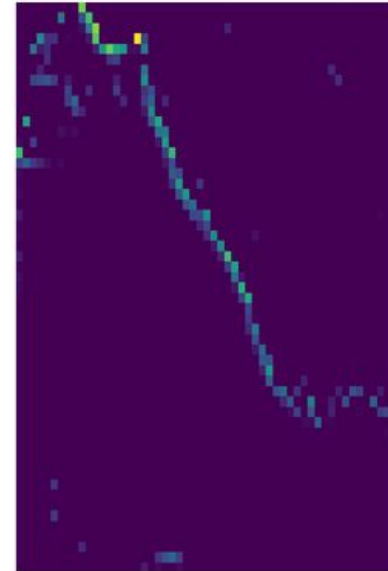
Input image



Filter 31



Filter 31 on input image



Going deeper

Residual Networks (ResNets)

A close-up shot of Leonardo DiCaprio in a dark suit, white shirt, and patterned tie. He has a serious, intense expression and is looking slightly to his right. The lighting is warm and focused on his face. Another person's head and shoulder are visible in the foreground on the right, partially obscuring the view.

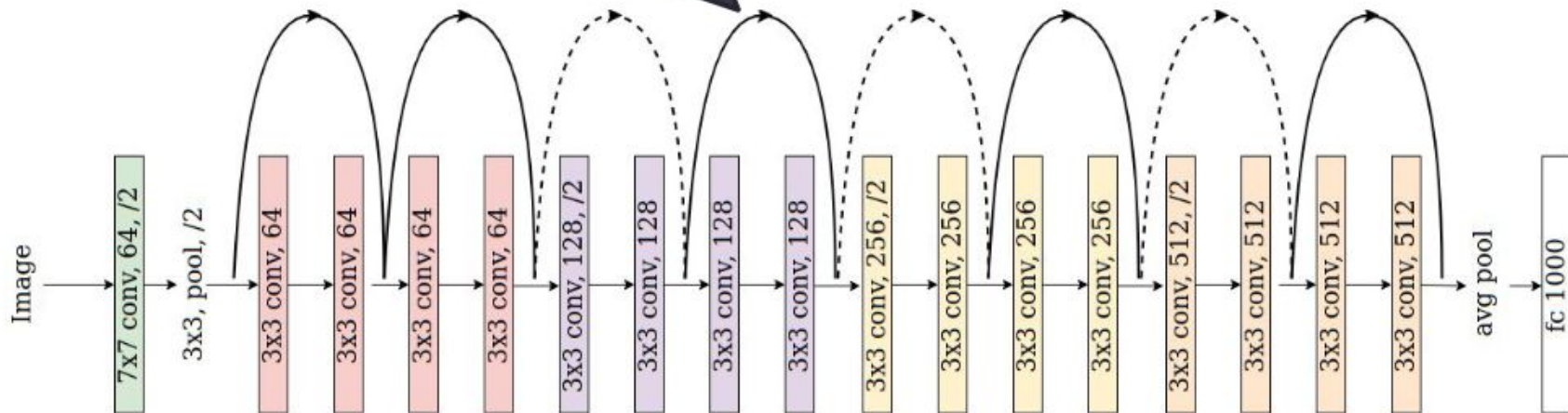
WE NEED TO GO

DEEPER

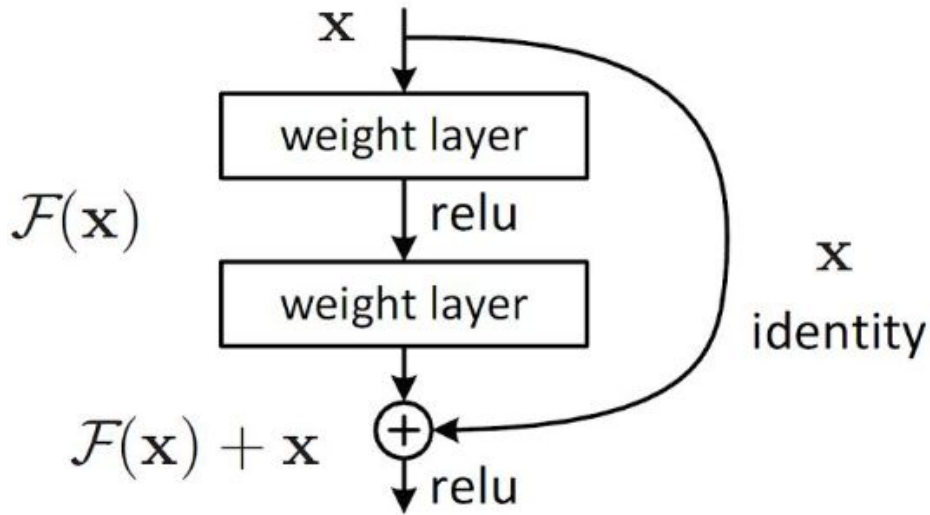
ResNet (2015)

```
torchvision.models.resnet18()  
...  
torchvision.models.resnet152()
```

Skip connections
(Residual blocks)



Residual blocks (“skip connections”)

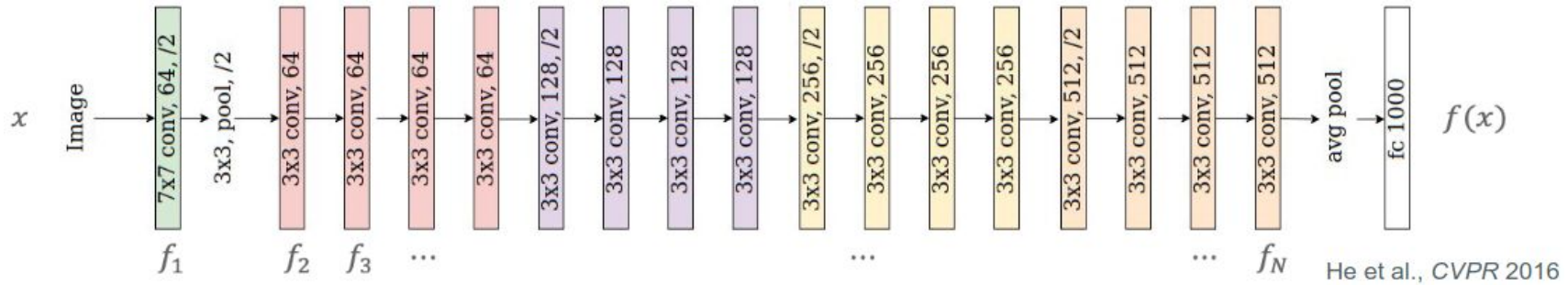


Each block learns to approximate a residual function

Better gradient flow because of skip connections

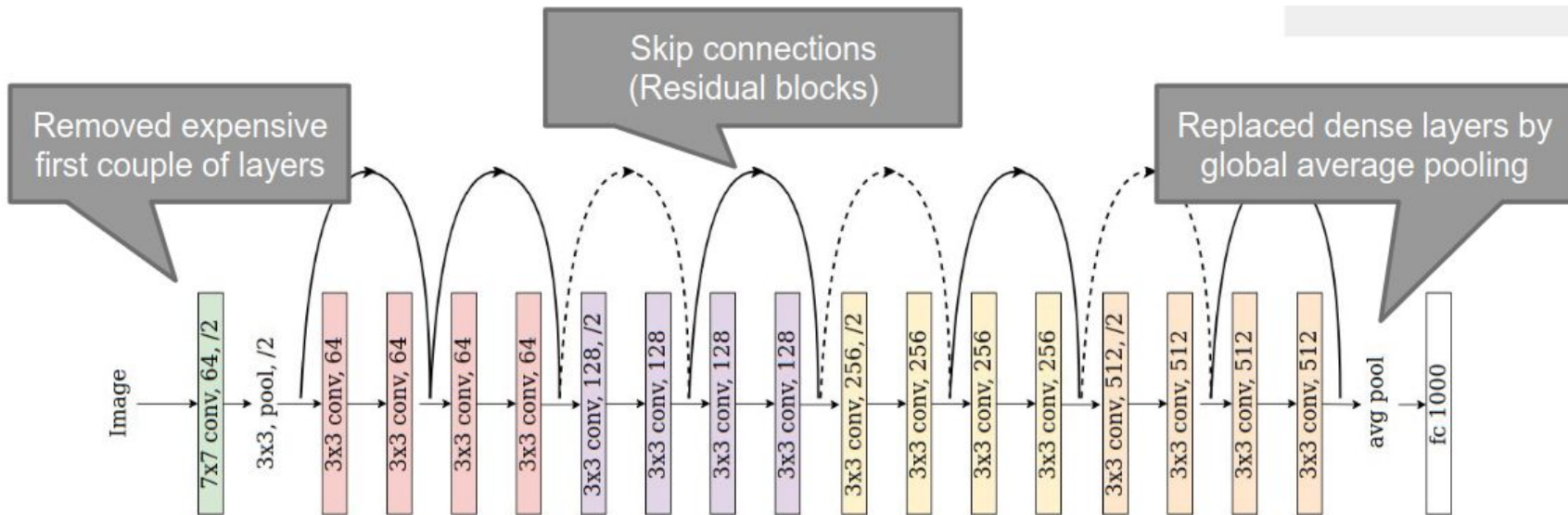
Skip connections avoid vanishing gradients

$$f(x) = f_1(f_2(\dots f_N(x))) \Rightarrow f'(x) = f'_1(f_2(\dots)) \cdot f'_2(\dots) \cdot \dots \cdot f'_N(x) \quad (\text{chain rule})$$

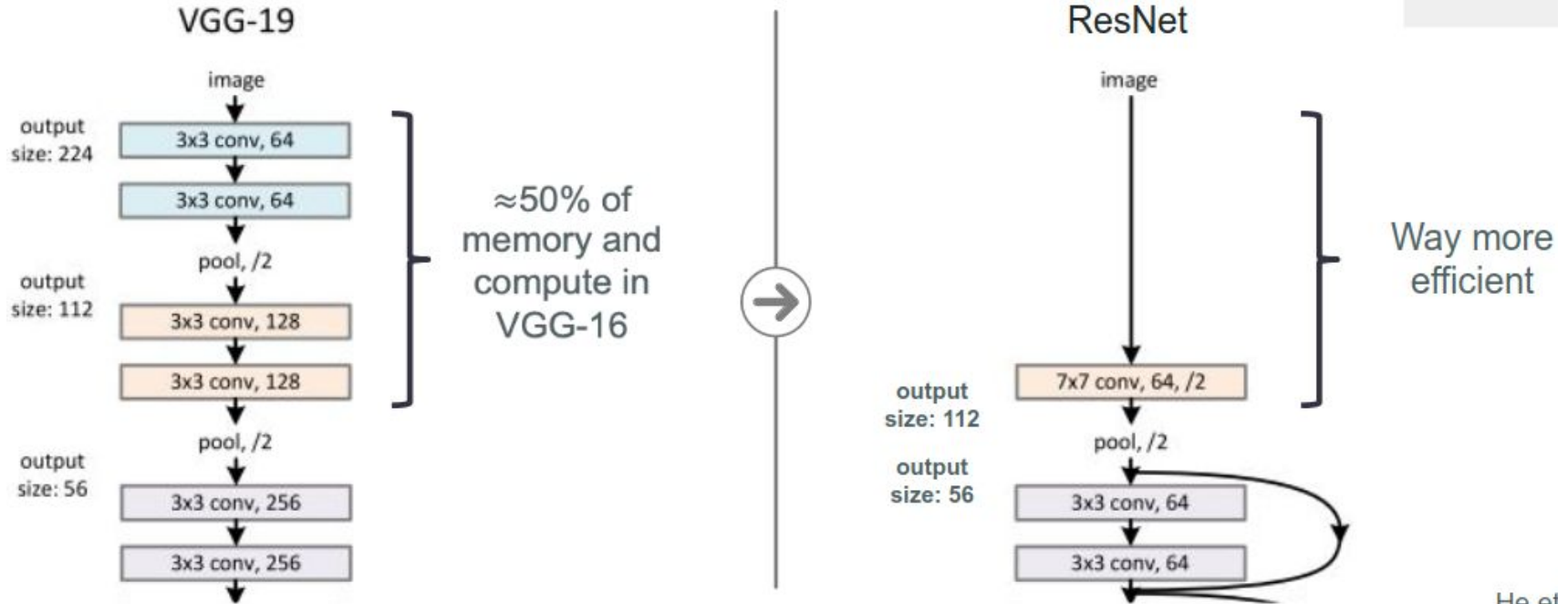


ResNet (2015)

```
torchvision.models.resnet18()  
...  
torchvision.models.resnet152()
```



ResNet: Remove expensive early layers



He et



Saturdays.AI
Kigali

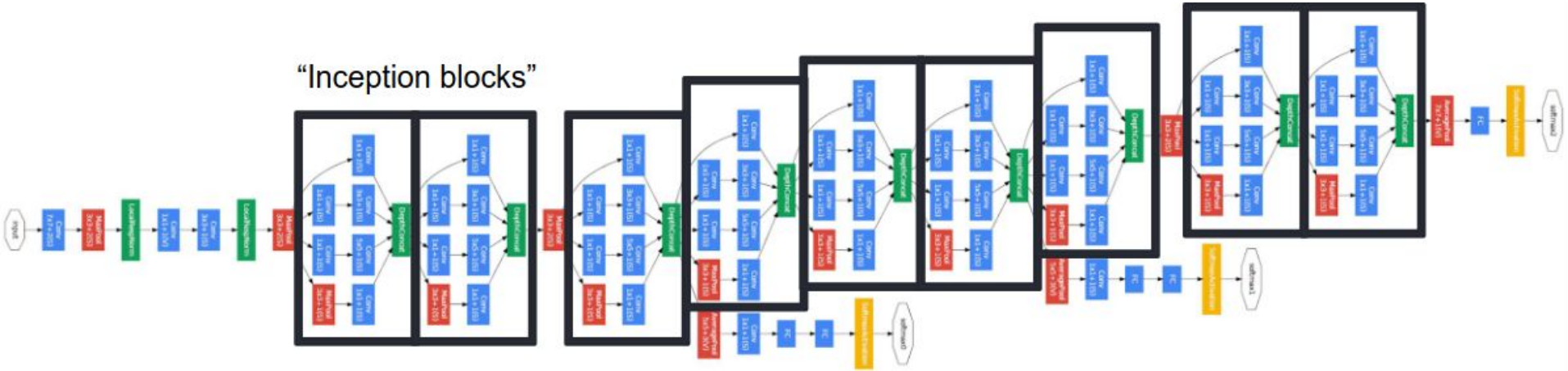
Improving efficiency

Part 1: Inception + ResNeXt



Saturdays.AI
Kigali

Inception / GoogLeNet (2014/15)



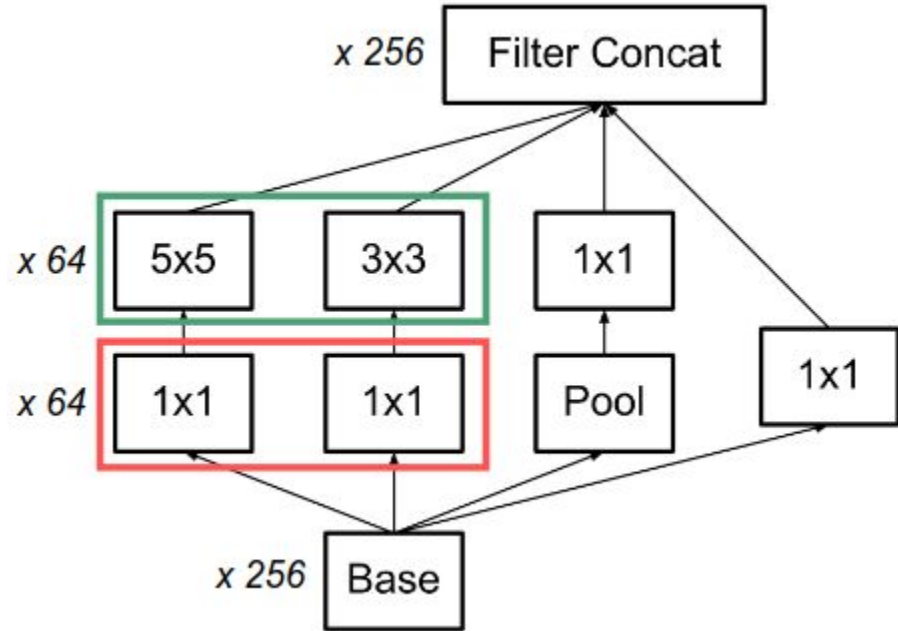
Inception v1: efficiency by 1 x 1 convolutions

```
torchvision.models.googlenet()
```

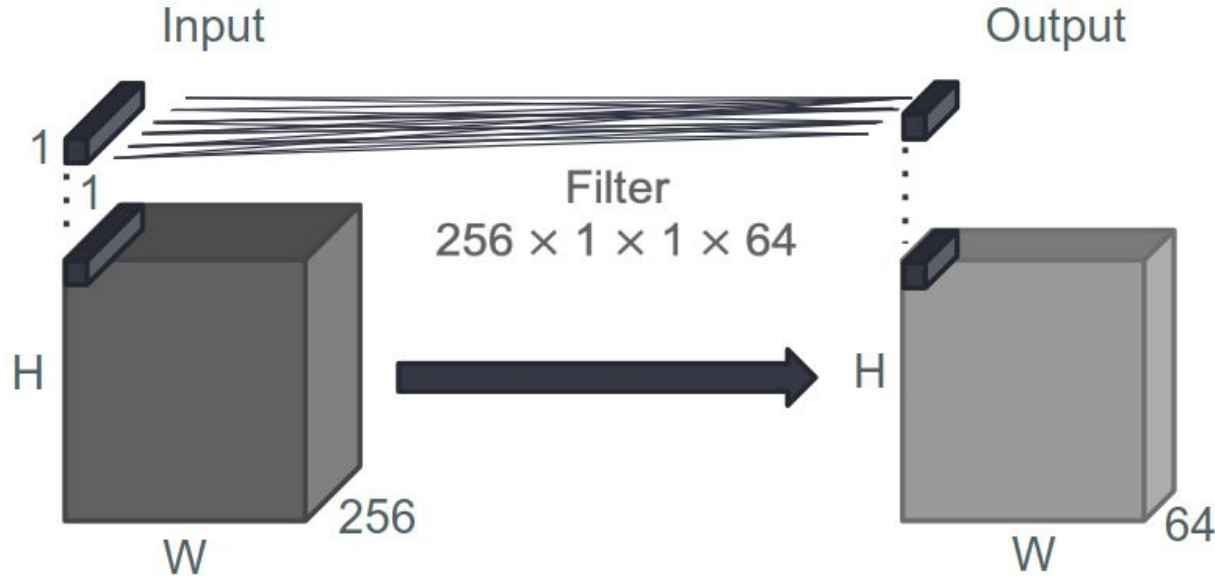
A single "Inception Module"

Spatial Processing

Reducing number of feature maps



1 x 1 convolutions



No spatial
processing

Reduces number
of channels

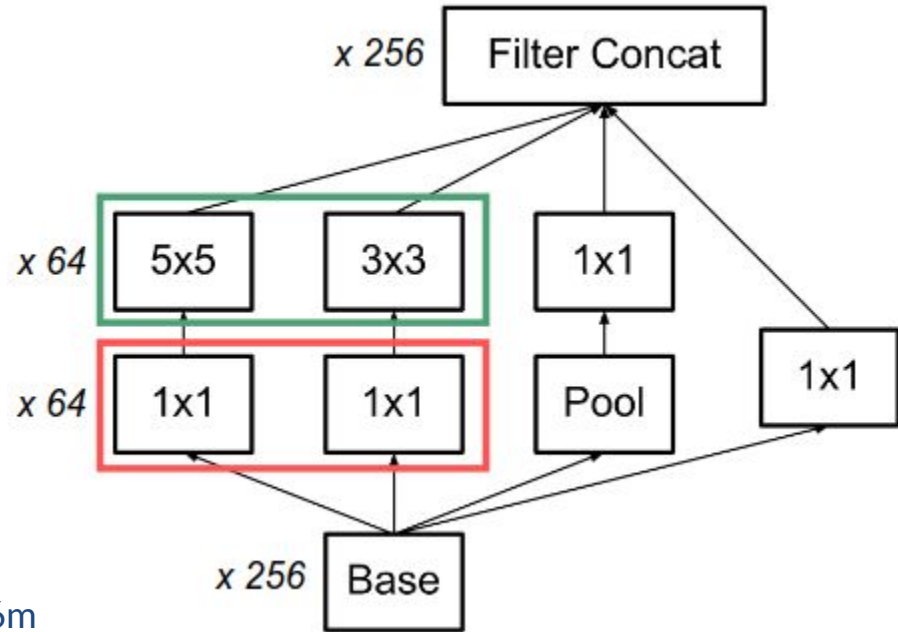
Inception v1: efficiency by 1 x 1 convolutions

```
torchvision.models.googlenet()
```

A single "Inception Module"

Spatial Processing

Reducing number of feature maps



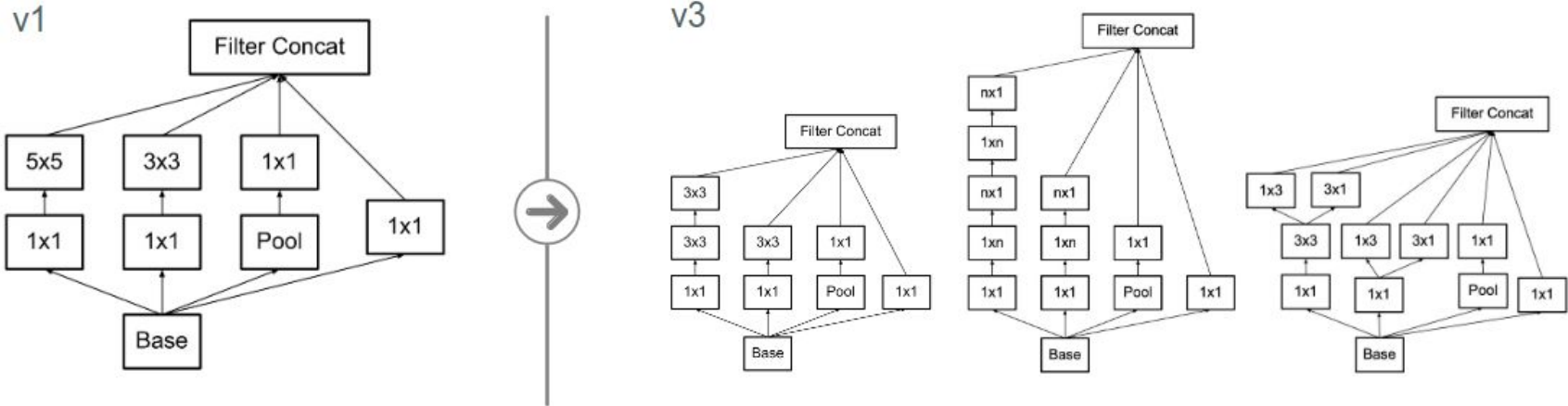
parameters

Standard convolution: $256 \times 5 \times 5 \times 256 = 1.6\text{m}$

Inception branch: $256 \times 64 + 64 \times 5 \times 5 \times 64 = 120\text{k}$

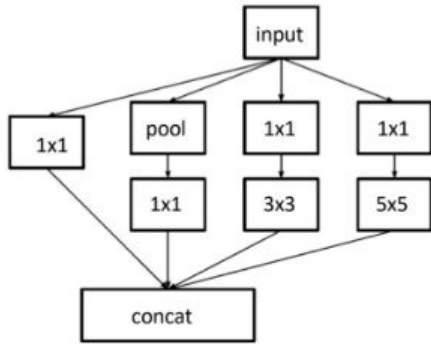
Inception v3: efficiency and bottlenecks

`torchvision.models.inception_v3()`

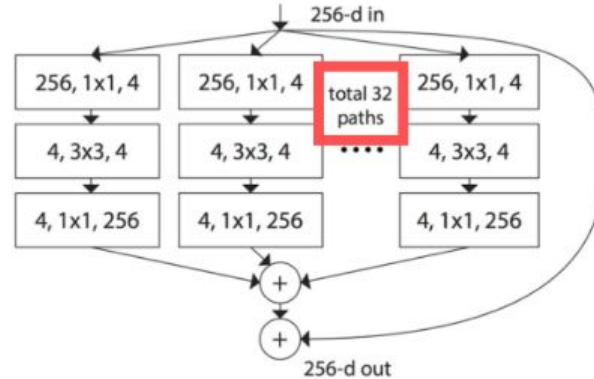


ResNeXt: combining Inception and ResNet

```
torchvision.models.models.resnext50_32x4d()
```



Inception:
heterogeneous multi-branch



ResNeXt:
uniform multi-branch

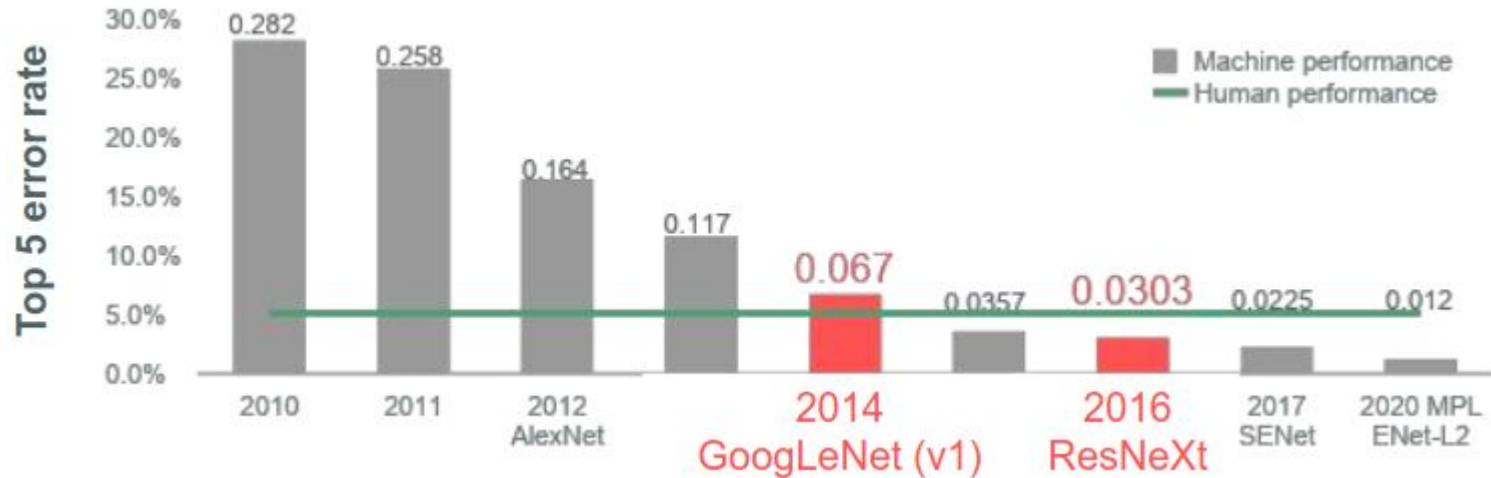
Compression
Spatial processing
Channel mixing



Inception-v1 (2014) + ResNeXt (2016): Results

6.7% top-5 error rate in 2014, 3.0% in 2016!

Top-5 error rate of human expert (Andrej Karpathy) was ~5%



Improving efficiency

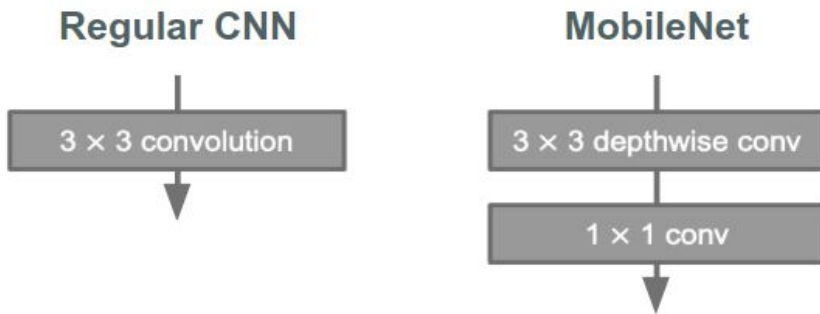
Part 2: MobileNet / depthwise separable convolutions



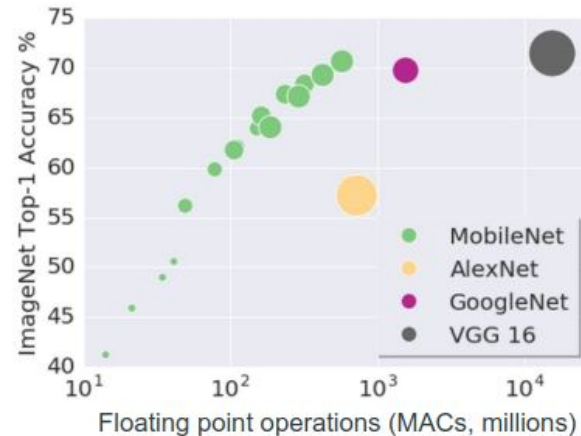
Saturdays.AI
Kigali

MobileNet (2017): pushing efficiency

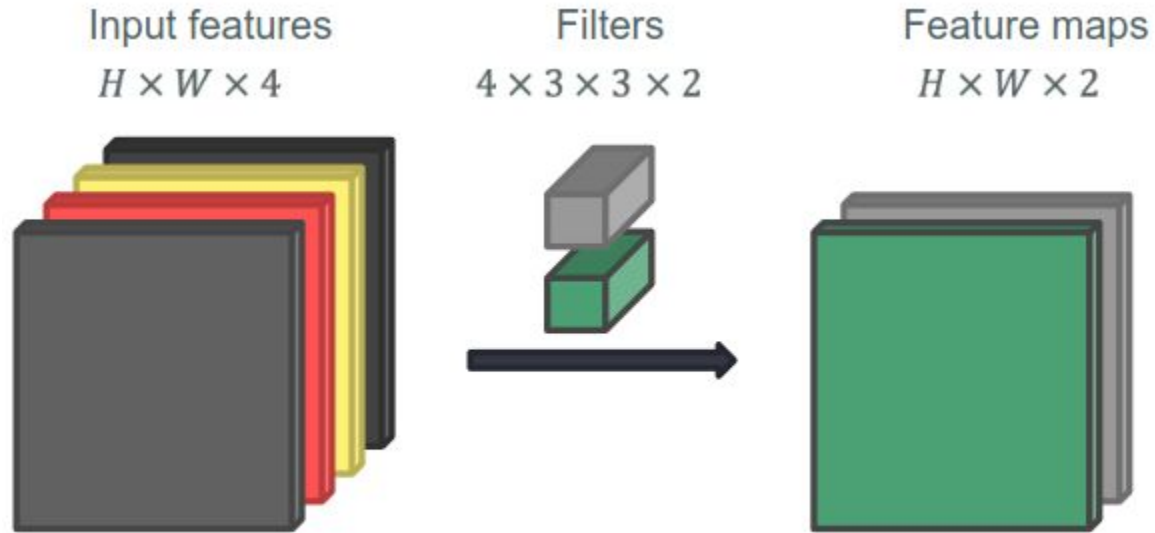
```
torchvision.models.mobilenet_v2()
```



Same accuracy as VGG, >100 frames/sec on iPhone 7

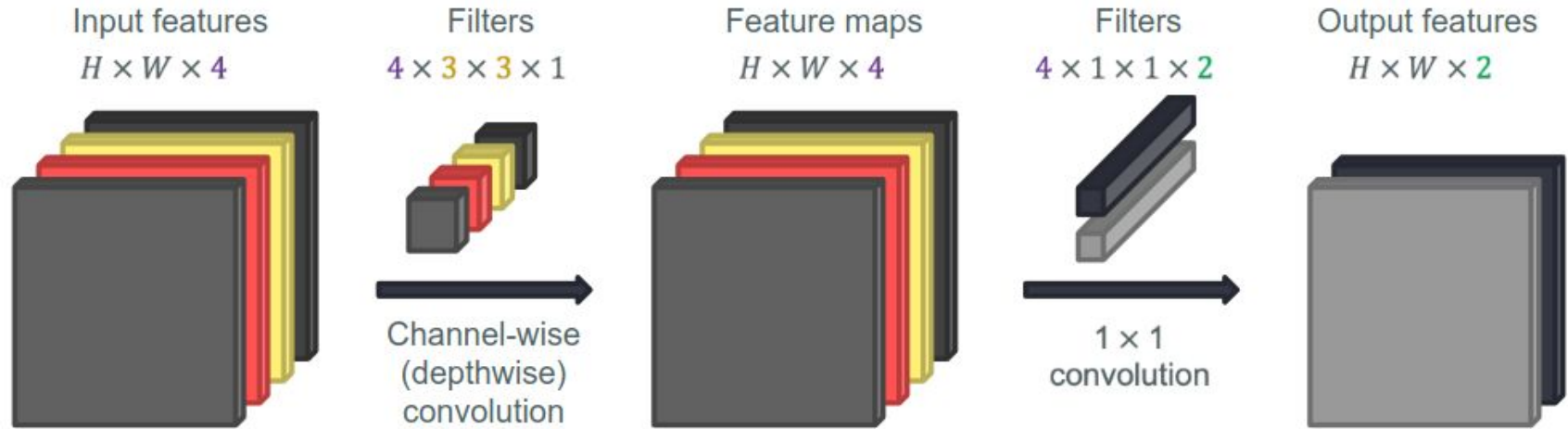


Recap: regular convolution layer



Regular convolution: $4 \cdot 3 \cdot 3 \cdot 2 = 72$ parameters

Depthwise separable convolution



Depthwise separable: $4 \cdot 3 \cdot 3 + 4 \cdot 2 = 44$ parameters

Regular convolution: $4 \cdot 3 \cdot 3 \cdot 2 = 72$ parameters

$O(MK^2 + MN)$ parameters

$O(MK^2N)$ parameters

Depthwise separable convolution



Depthwise separable: $512 \cdot 3 \cdot 3 + 512 \cdot 512 = 267\text{k parameters}$

Regular convolution: $512 \cdot 3 \cdot 3 \cdot 512 = 2.4\text{m parameters}$

Transfer learning

Building upon pre-trained CNNs

This is us



IMAGENET

Standing on
the shoulders
of giants

Image source: <https://www.cgtrader.com/3d-models/character/man/cartoon-giant>

Transfer learning: motivation

Suppose you want to build an app to recognize flowers...



alpine sea holly



buttercup



fire lily



cape flower



anthurium



californian poppy



artichoke



camellia



azalea



canna lily



fritillary



clematis

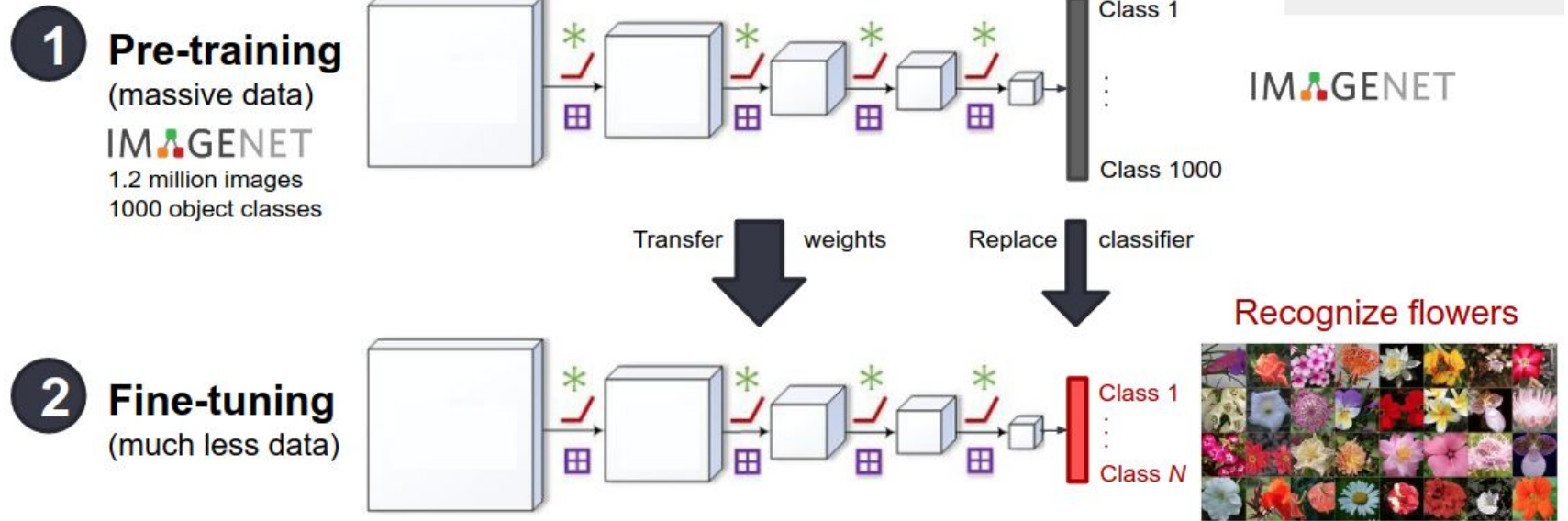
Training a convolutional
net from scratch won't
work well

1000 images
100 classes



Saturdays.AI
Kigali

Transfer learning: the idea



Transfer learning: what should we train?

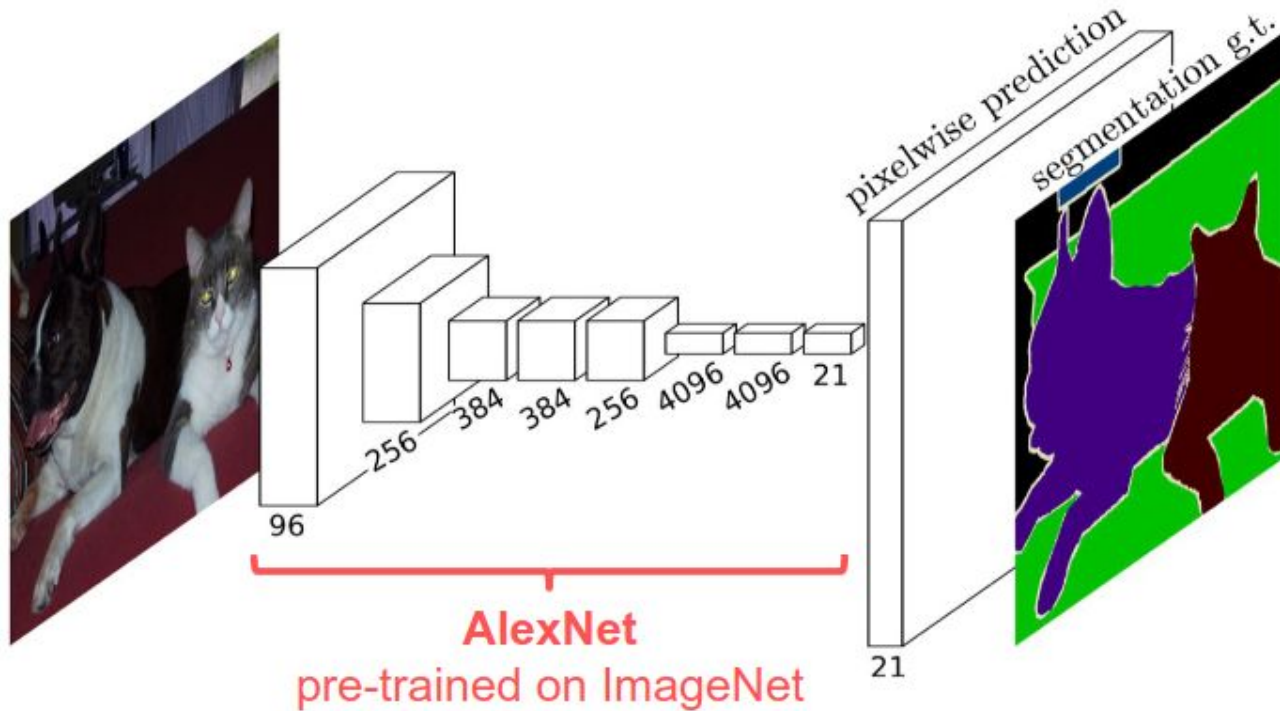
Option 1: Train only classification layer, freeze backbone
(sometimes referred to as the “linear evaluation protocol”)

- Fast & simple

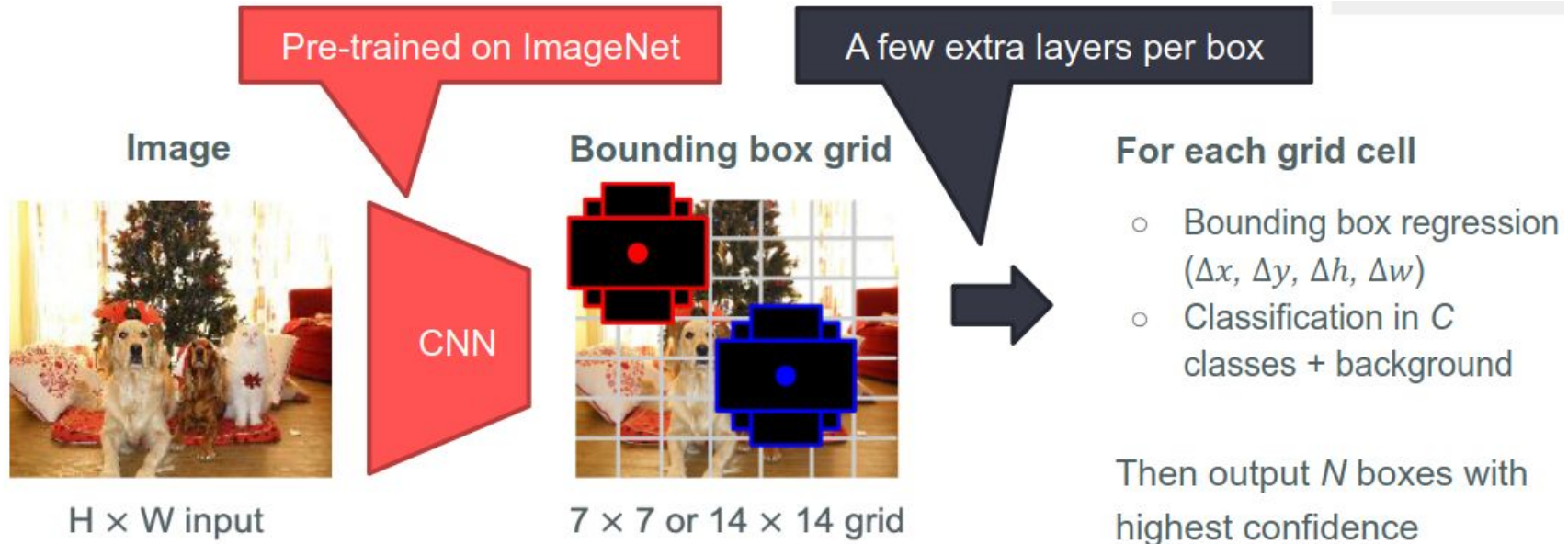
Option 2: Train classification layer, fine-tune backbone at the same time

- Slower, but can adapt feature extraction to dataset statistics

Example: semantic segmentation



Single-stage object detectors: SSD, YOLO, ...



Pre-training on ImageNet is everywhere ...

Pose estimation

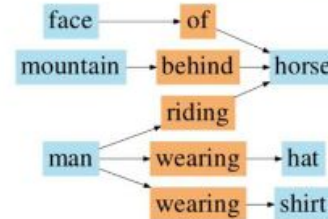
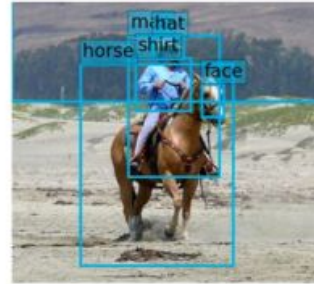


Image captioning



"two young girls are playing with
lego toy."

Scene graph prediction



...

Many,
many
more

TYU: Convolution

Assume that the number of filters is less than number of input channel, Which of the following will reduce the number of dimensions of input X

- A. 1x1 CONV, 3 filters, no padding
- B. 3x3 CONV, 3 filters, no padding
- C. 3x3 POOL, 3 filters, no padding
- D. All of the above



Practice

Train a CNN

CNN for smile classification

CNN



Saturdays.AI
Kigali

Break

Practice

We give you code. What happens?

Run. Learn. Repeat.



Saturdays.AI
Kigali

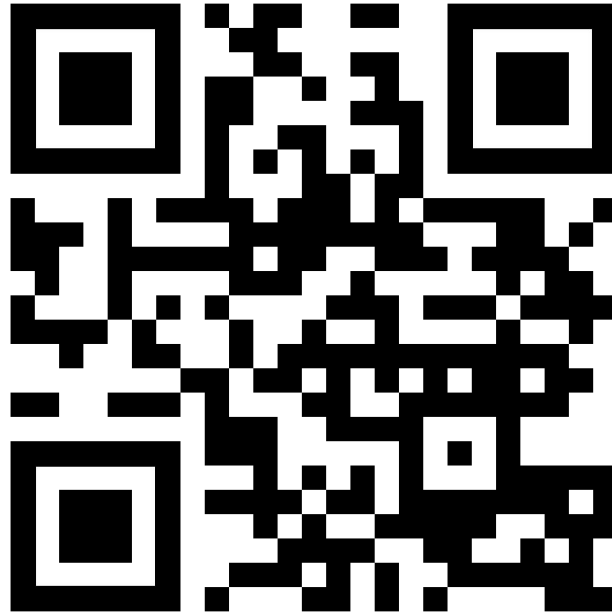
Challenges & Next steps!



Best part of the day!



Kahoot!
kahoot.it



**Any
questions?**



Saturdays.AI
Kigali



Saturdays.AI
Kigali

THANKS



kigali@saturdays.ai



coming soon



coming soon